

Reviewer Pack — Minimal Local Cohomology Rank and Communication Complexity f...

Entry a80c41a85b9f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 05:49:51 UTC

Minimal Local Cohomology Rank and Communication Complexity for XOR

Entry ID: a80c41a85b9f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 05:49:51 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology: Local Cohomology **Field B** (complexity object): Communication Complexity: XOR

Statement:

For all n -ary XOR functions with n inputs, the Resolution proof length for the corresponding Tseitin formula is $\Theta(2^{(\log_2(n) + \log_2(H(n)))})$, where $H(n)$ is the minimal local cohomology rank of the associated simplicial complex.

Rationale (proposer's reasoning):

Local cohomology provides a measure of algebraic complexity, and its properties might reveal inherent structure in computational problems like XOR. If this conjecture holds, it would imply a connection between algebraic topology and communication complexity that could potentially lead to new proof systems or better understanding of the hardness of XOR functions.

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7988af0093e41943

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean Resolution proof length of Tseitin formulas for random XOR functions with n inputs is within ± 3 from $\Theta(2^{(\log_2(n) + \log_2(H(n)))})$ over 30 seeds, and falsified if any seed produces a deviation greater than 3 from this expected value.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local cohomology rank" AND "communication complexity" AND XOR - "Tseitin formula" AND Resolution proof length" AND minimal local cohomology rank" - "algebraic topology" AND simplicial complex" AND communication complexity XOR function

Top relevant hits considered: - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/2006.04124v1] On the Complexity of Branching Proofs - [http://arxiv.org/abs/2402.00542v1] Hardness of Random Re-ordered Encodings of Parity for Resolution and CDCL - [http://arxiv.org/abs/2506.12921v2] Shortest Paths in a Weighted Simplicial Complex - [http://arxiv.org/abs/2401.00699v3] Quantum walk on simplicial complexes for simplicial community detection - [http://arxiv.org/abs/1411.5306v3] The Simplicial EHP Sequence in A1-Algebraic Topology

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.8s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 10 # Start with a small value and increase if needed

    def generate_xor_function(n):
        return [random.randint(0, 1) for _ in range(n)]

    def tseitin_formula(xor_func):
        n = len(xor_func)
        literals = list(range(-n, 0)) + list(range(1, n+1))
        clauses = []

        # Base case
        for i in range(n):
            clauses.append([literals[i], literals[n+i]])

        # XOR logic
        for i in range(n):
            clause = [-literals[2*n+i]]
            for j in range(n):
                if xor_func[j] == (i + 1) % n:
```

```

        clause.append(literals[j])
    else:
        clause.append(-literals[j])
    clauses.append(clause)

    return clauses

def resolution_proof_length(clauses):
    # Simplified version of Resolution proof length calculation
    return len(clauses)

def minimal_local_cohomology_rank(n):
    # Placeholder function for minimal local cohomology rank
    # This is a dummy implementation and should be replaced with actual computation
    return n

total_length = 0
instances_tested = 0

for _ in range(30): # Ensure at least 30 instances per seed
    xor_func = generate_xor_function(n)
    clauses = tseitin_formula(xor_func)
    length = resolution_proof_length(clauses)
    total_length += length
    instances_tested += 1

mean_length = Fraction(total_length, instances_tested)
expected_length = 2**(math.log2(n) + math.log2(minimal_local_cohomology_rank(n)))

conjecture_holds = abs(mean_length - expected_length) <= 3
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Resolution proof length",
    "metric_value": mean_length,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)] # Default to first 3 seeds

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(res["metric_value"] for res in results) / len(results)
    std_dev = math.sqrt(sum((res["metric_value"] - mean_length)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_dev} support_fraction={support_fraction}")

```

```

elif any(not res["conjecture_holds"] for res in results) and support_fraction >= 0.8:
    first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a6c340cb.py", line 86, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a6c340cb.py", line 61, in run_trial
    clauses = tseitin_formula(xor_func)
               ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a6c340cb.py", line 37, in tseitin_formula
    clause = [-literals[2*n+i]]
               ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the mean Resolution proof length of Tseitin formulas for random XOR functions meets the support condition. | next: Investigate and fix the crash in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15962
2	propose	ollama_remote	glm4:latest	0	0	11811
3	preregistration	ollama_remote	glm4:latest	0	0	10299
4	novelty	ollama_remote	glm4:latest	0	0	10425
5	novelty	ollama_remote	glm4:latest	0	0	8990
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15591
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9087

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12328
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9741
10	judge	ollama_remote	glm4:latest	0	0	17091

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 121326 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/a80c41a85b9f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a80c41a85b9f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a80c41a85b9f.tar.gz (if generated)